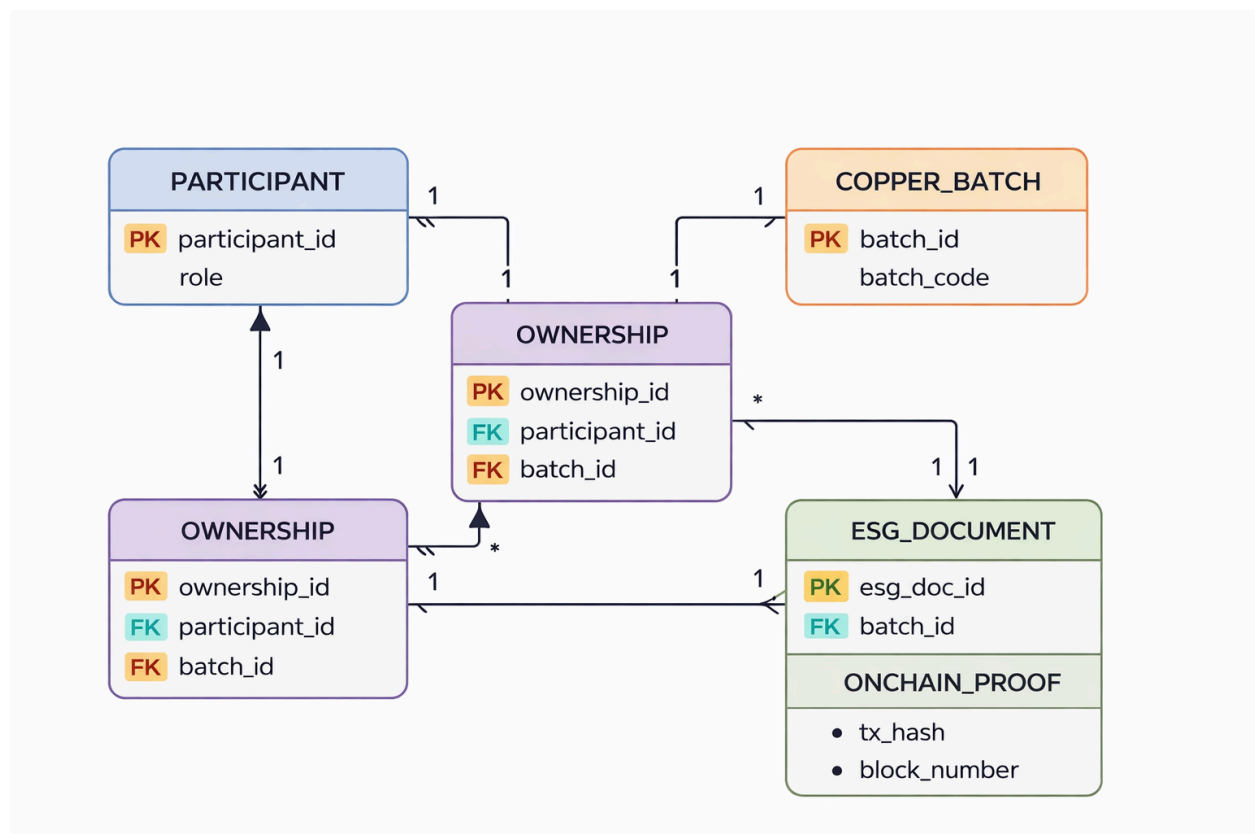


POC architecture for the Copper RWA project:

1. Core Definitions

- **Digital Twin:** A blockchain-based representation of a physical copper batch, containing an immutable record of its origin, weight, and grade.
- **ESG Hash:** A SHA-256 cryptographic fingerprint of off-chain environmental and social data (e.g., carbon emissions, labor certifications) stored on-chain to prevent data tampering.
- **State Machine:** A logic flow within the smart contract that restricts copper batches to a specific sequence of statuses (e.g., MINED → IN_TRANSIT).
- **Permissioned Node:** A network participant that requires explicit authorization to join, ensuring the data remains within the enterprise consortium.



2. Smart Contract Logic (Solidity)

The following contract defines the data structure and access control for the copper batch.

```
``solidity

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.17;

contract CopperRWA {
    // Definitive states for the physical asset
    enum AssetStatus { Mined, Processed, InTransit, Exported, Delivered }

    struct CopperBatch {
        string batchId;    // Unique Identifier from ERP
        uint256 weight;    // Weight in Metric Tons
        bytes32 dataHash;  // ESG & Logistics metadata hash
        address custodian; // Current entity responsible for the batch
        AssetStatus status; // Current stage in lifecycle
    }

    mapping(string => CopperBatch) public registry;
    mapping(address => bool) public isAuthorized;

    // Restricts function execution to registered miners/logistics providers
    modifier onlyAuthorized() {
        require(isAuthorized[msg.sender], "Unauthorized entity");
        _;
    }

    function createBatch(string memory _id, uint256 _weight, bytes32 _hash) public onlyAuthorized {
        registry[_id] = CopperBatch(_id, _weight, _hash, msg.sender, AssetStatus.Mined);
    }

    function updateStatus(string memory _id, AssetStatus _nextStatus, bytes32 _newHash) public
    onlyAuthorized {
        CopperBatch storage batch = registry[_id];
        batch.status = _nextStatus;
        batch.dataHash = _newHash;
    }
}
```

3. Data Ingestion & Hashing (Node.js)

The backend manages the link between the raw ESG data and the blockchain.

- **Data Normalization:** Converts varied IoT and ERP inputs into a standard JSON format.
- **Integrity Verification:** Uses the `crypto` library to generate a hash that anchors the data.

```
```JavaScript
```

```
const crypto = require('crypto');

/**
 * Creates a deterministic hash of the ESG metadata.
 * @param {Object} metadata - The raw ESG and mining data.
 * @returns {string} - The hex-encoded SHA-256 hash.
 */
function createIntegrityHash(metadata) {
 const dataString = JSON.stringify(metadata, Object.keys(metadata).sort());
 return '0x' + crypto.createHash('sha256').update(dataString).digest('hex');
}

// Example ESG Payload
const currentEsgData = {
 carbonIntensity: "2.4t",
 waterRecyclingRate: "85%",
 mineId: "CHI-092"
};

const onChainHash = createIntegrityHash(currentEsgData);
...

```

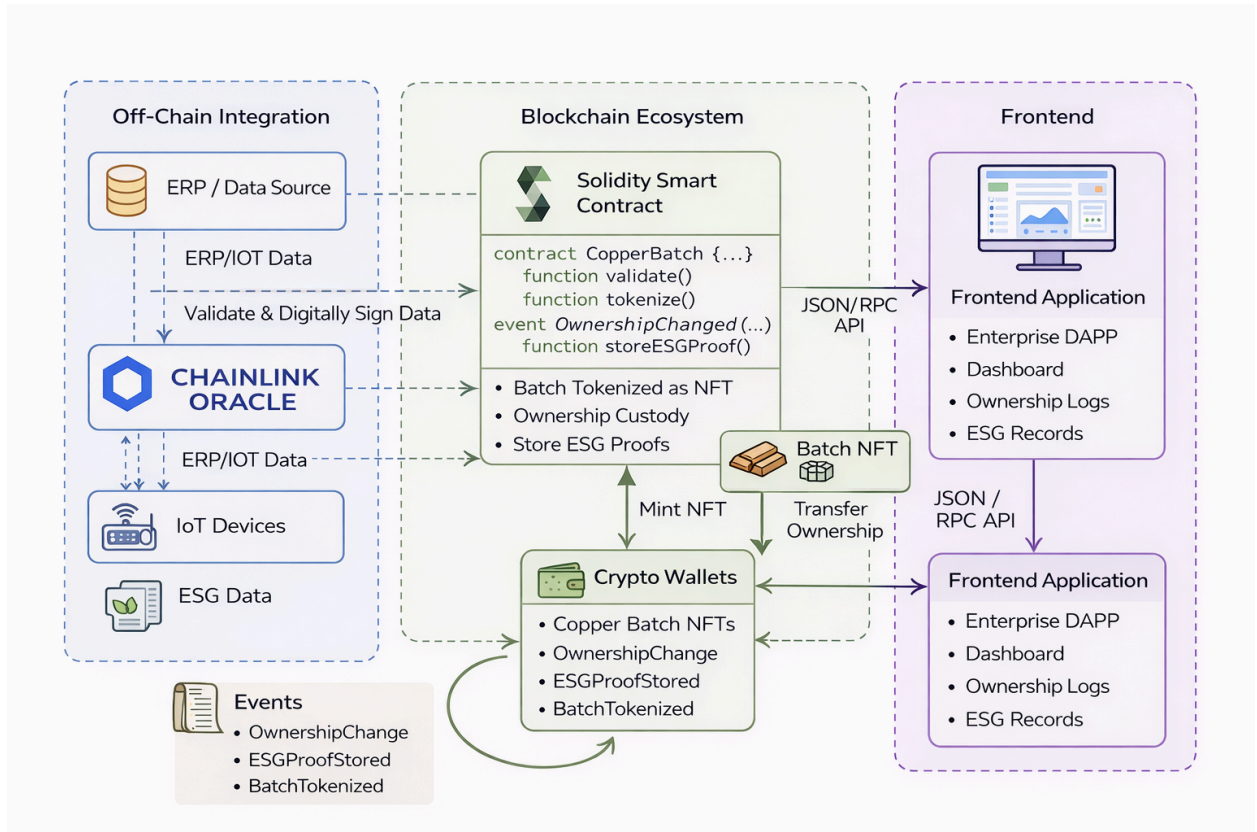
---

### 4. POC Components & Roles

The system operates through specific interactions between defined roles.

- **Mining Node:** Initiates the "Digital Twin" by inputting initial weight and grade.
- **ESG Oracle:** A service that periodically updates the `dataHash` with new environmental metrics.
- **Logistics Provider:** Updates the `AssetStatus` as the copper moves from the mine to the shipping port.

- **Regulatory Auditor:** A read-only role that verifies that the on-chain `dataHash` matches the provided off-chain ESG documents.



## 5. Security & Verification Layer

- **Digital Signatures:** Every state change requires an EIP-712 signature from the custodian's private key.
- **Audit Trail:** The blockchain provides a time-stamped history of every `AssetStatus` change, which cannot be backdated or altered.
- **Off-chain Storage:** Sensitive documents (PDFs, invoices) are stored in encrypted S3 buckets, with only the hash and URI stored on the ledger.